

初等数论基础

Hex

NOI大纲

入门级

3.初等数论

- 【3】 整除、因数、倍数、指数、质（素）数、合数
- 【3】 取整
- 【3】 模运算与取余
- 【3】 整数唯一分解定理
- 【3】 辗转相除法（欧几里得算法）
- 【4】 素数筛法：埃氏筛法与线性筛法

提高级

2.初等数论

- 【5】 同余式
- 【7】 欧拉定理和欧拉函数
- 【7】 费马小定理
- 【7】 威尔逊定理
- 【7】 裴蜀定理
- 【7】 模运算意义下的逆元
- 【7】 扩展欧几里得算法
- 【7】 中国剩余定理

NOI级

1.初等数论

- 【8】 原根和指数
- 【8】 大步小步（Baby Step Giant Step, BSGS）算法
- 【9】 狄利克雷（Dirichlet）卷积
- 【10】 二次剩余
- 【10】 二次同余式

整除与约数

对于正整数 n, m ，如果存在正整数 k 使得 $n = km$ ，则称 m 整除 n ，记作 $m|n$ 。

称 m 是 n 的约数， n 是 m 的倍数。

例子： $12 = 3 \times 4$ ，3和4能整除12，因此3和4都是12的约数，12是3和4的倍数。

需要注意，由于 $12 = (-3) \times (-4)$ ，认为-3和-4也同样能整除12同样是12的约数。

模算术(%)

算法竞赛中，数论部分的一个重要组成部分就是 **模算术**，也就是在某一模数下进行各种整数运算。模算术常见于各类问题中，而不仅仅局限于数论部分。

很多问题的实际答案可能非常大，超过了常见的整型变量的存储范围。此时，为了避免引入大整数运算和输出长数字，题目常常要求对答案取模后输出。这就要求熟练掌握各类模算术技巧。

在推式子中，如果出现取模相关运算，一般转化为熟悉的四则运算再进行操作。

$$a \bmod b = a - \left\lfloor \frac{a}{b} \right\rfloor \times b$$

abc452_e

给你一个长度为 N 的正整数序列 $A = (A_1, \dots, A_N)$ ，还有一个长度为 M 的正整数序列 $B = (B_1, \dots, B_M)$ 。

求 $\sum_{i=1}^N \sum_{j=1}^M A_i \cdot B_j \cdot (i \bmod j)$ 对 998244353 取模后的结果。

调和级数

- 调和级数的定义为 $H_n = \sum_{k=1}^n \frac{1}{k}$ 。
- 例如，当 $n = 5$ 时， $H_5 = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} \approx 2.2833$
- 关于调和级数有如下结论：

$$H_n = \ln n + \gamma + \varepsilon_k$$

- 在信息学中， n 的范围经常在 $10^5 \sim 10^7$ 之间，其 $\ln n$ 的值在 $10 \sim 17$ 之间。

质数与合数

- 质数又称素数，指的是不能被1以及自身以外的数，整除的数字。
- 相反，能被其它数整除的数，称为合数。
- 注意，1既不是质数，也不是合数。（定义）
- 例：11是质数。12=2*6或者3*4，所以12是合数。
- 判断一个数是不是质数，可以 $O(\sqrt{n})$ 判定。
- 质数分布：在1~ n 中，质数个数约为 $\frac{n}{\ln n}$ 个。

唯一分解定理

对于大于1的正整数 n ，都可以唯一地表示为有限个素数的乘积。

$$n = p_1^{a_1} \times p_2^{a_2} \times \cdots \times p_k^{a_k}$$

- $p_1, p_2, \dots, p_k$ 是互不相同的素数， $a_1, a_2, \dots, a_k$ 是正整数指数。
- 例如 $60 = 2^2 \times 3^1 \times 5^1$ ， $1400 = 2^3 \times 5^2 \times 7^1$
- 求一个数唯一分解的复杂度是？

最大公约数

最大公约数即为 Greatest Common Divisor, 常缩写为 gcd.

一组整数的公约数, 是指同时是这组数中每一个数的约数的数。 ± 1 是任意一组整数的公约数。

一组整数的最大公约数, 是指所有公约数中最大的一个。

对不全为0的整数 a, b , 将其最大公约数记为 $\gcd(a, b)$, 也可简写为 (a, b) 。

对不全为0的整数 $a_1, a_2, \dots, a_n$, 将其最大公约数记为 $\gcd(a_1, a_2, \dots, a_n)$, 也可简写为 $(a_1, a_2, \dots, a_n)$ 。

如何求最大公约数?

欧几里得算法

已知两个数 a, b ，求最大公约数 $\gcd(a, b)$ 。

不妨设 $a > b$ 。

如果 b 是 a 的约数，那么 b 就是两者的最大公约数。

否则，令 $a = bk + r$ ，其中 $r < b$ ，需要证明 $\gcd(a, b) = \gcd(b, r)$ 。

提示：证明 a, b 的公约数与 b, r 的公约数完全一样。

设 $d|a, d|b$ ，则 $r = a - bk$ 得到 $\frac{r}{d} = \frac{a}{d} - \frac{b}{d}k$ 。

由于右边的式子一定是整数，那么 $d|r$ ，因此 a, b 的公约数与 b, r 的公约数完全一样。

递归求解：

```
int gcd(int a, int b){  
    return b == 0 ? a : gcd(b, a%b);  
}
```

多个数的最大公约数？

互质

若正整数 a, b 满足 $\gcd(a, b) = 1$ ，则称 a 与 b 互质。记作 $a \perp b$

若 $\gcd(a_1, a_2, \dots, a_n) = 1$ ，则称 $a_1, a_2, \dots, a_n$ 互质。

多个整数互质，不一定两两互质。

例如6、10和15互质，但是任意两个都不互质。

朴素筛

- 对于任意一个大于 1 的正整数 x ，其倍数一定是合数。
- 我们从小到大考虑每个数，对于数 x ，将比 x 大的倍数都标记为合数。
- 在结束时，没有被标记的数就是质数。
- 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16
- 对于每个 x 而言，需要考虑的倍数的数量大致为 $\frac{n}{x}$ 。
- 因此，整体的时间复杂度为：

$$\frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \cdots + \frac{n}{n} = n \left(\frac{1}{2} + \frac{1}{3} + \frac{1}{4} \cdots + \frac{1}{n} \right) \approx n \ln n$$

朴素筛

- 当 $n = 10^6$ 时需要 50ms 左右；当 $n = 10^7$ 时需要 600ms 左右。

```
// 朴素筛法
bool vis[Maxn];
void simple(int n){
    // 枚举 2 ~ n
    for(int i=2;i<=n;i++){
        /* 当 i = 2 时, 枚举 4, 6, 8, 10, ...
           当 i = 3 时, 枚举 6, 9, 12, 15, ...
           当 i = 4 时, 枚举 8, 12, 16, 20, ...
           ... */
        for(int j=i+i;j<=n;j+=i)
            // 打上标记
            vis[j] = true;
    }
}
```

1, 2, 3, ~~4~~, 5, ~~6~~, 7, ~~8~~, 9, ~~10~~, 11, ~~12~~, 13, ~~14~~, 15, ~~16~~

埃氏筛

1, 2, 3, ~~4~~, 5, ~~6~~, 7, ~~8~~, 9, ~~10~~, 11, ~~12~~, 13, ~~14~~, 15, ~~16~~

- 注意到，所有4的倍数都已经被2的倍数给筛去。
- 被标记过的数不需要再额外遍历其倍数。
- 等同于：只需要筛去质数的倍数即可。

1, 2, 3, ~~4~~, 5, ~~6~~, 7, ~~8~~, ~~9~~, ~~10~~, 11, ~~12~~, 13, ~~14~~, ~~15~~, ~~16~~

- 2的倍数： 3的倍数： 5的倍数： 的倍数：

朴素筛： 1, 2, 3, ~~4~~, 5, ~~6~~, 7, ~~8~~, ~~9~~, ~~10~~, 11, ~~12~~, 13, ~~14~~, ~~15~~, ~~16~~

埃氏筛

- 埃氏筛只需要遍历质数的倍数即可，由于质数的密度为 $\frac{1}{\ln n}$ ，平均每 $\ln n$ 个数才需要遍历一次倍数，因此其时间复杂度大致为 $O(n \ln \ln n)$ 。
- 当 $n = 10^7$ 仅需 150ms 左右。

```
// 埃氏筛
bool vis[Maxn];
void Eratosthenes(int n){
    // 枚举 2 ~ n
    for(int i=2; i<=n; i++){
        // 只需要遍历质数
        if( !vis[i] )
            for(int j=i+i; j<=n; j+=i)
                vis[j] = true;
    }
}
```

欧拉筛

- 有没有比埃氏筛还快的筛法？
- 欧拉筛也叫线性筛，保证了每个数仅被自己**最小的质因数筛去**。
- 枚举数字 i 时，再从小到大枚举已经筛出来的质数 p_j ，将 $i \times p_j$ 标记为合数。
- 当 p_j 是 i 的因数时在标记后跳出枚举质数的循环，每个被筛掉的数的最小质因数就是 p_j

```
int prim[Maxn], tol, minp[Maxn];
bool vis[Maxn];
void Init(int n){
    for(int i=2;i<=n;i++){
        if( !vis[i] ) prim[++tol] = i, minp[i] = i;
        for(int j=1;j<=tol&& i*prim[j]<=n;j++){
            vis[i*prim[j]] = true;
            minp[i*prim[j]] = prim[j];
            if( i%prim[j] == 0 ) break;
        }
    }
    for(int i=2;i<=40;i++) printf("i = %d, minp = %d\n", i, minp[i]);
}
```

预处理欧拉筛之后，
求唯一分解的复杂度是多少？

$$n = p_1^{a_1} \times p_2^{a_2} \times \dots \times p_k^{a_k}$$

每次除最小质数 $O(\sum_{i=1}^k a_i)$

存储最小质数的幂次和值 $O(k)$

i = 2,	minp = 2
i = 3,	minp = 3
i = 4,	minp = 2
i = 5,	minp = 5
i = 6,	minp = 2
i = 7,	minp = 7
i = 8,	minp = 2
i = 9,	minp = 3
i = 10,	minp = 2
i = 11,	minp = 11
i = 12,	minp = 2
i = 13,	minp = 13
i = 14,	minp = 2
i = 15,	minp = 3
i = 16,	minp = 2
i = 17,	minp = 17
i = 18,	minp = 2
i = 19,	minp = 19
i = 20,	minp = 2
i = 21,	minp = 3
i = 22,	minp = 2
i = 23,	minp = 23
i = 24,	minp = 2
i = 25,	minp = 5
i = 26,	minp = 2
i = 27,	minp = 3
i = 28,	minp = 2
i = 29,	minp = 29
i = 30,	minp = 2
i = 31,	minp = 31
i = 32,	minp = 2
i = 33,	minp = 3
i = 34,	minp = 2
i = 35,	minp = 5
i = 36,	minp = 2
i = 37,	minp = 37
i = 38,	minp = 2
i = 39,	minp = 3
i = 40,	minp = 2

更多筛法

- 有没有比欧拉筛还快的筛法？

算法	时间
判断质数	$O(\sqrt{n})$
Miller–Rabin 素性测试*	$O(k \ln n)$
朴素筛	$O(n \ln n)$
埃氏筛	$O(n \ln \ln n)$
欧拉筛	$O(n)$
杜教筛*	$O(n^{2/3})$
Min_25筛*	$O(\frac{n^{3/4}}{\ln n})$

Miller–Rabin: k 轮测试检测大数 n 是否为质数(n 可以比 10^{18} 还大, 但结果不一定正确)。

杜教筛/Min_25筛: 低于线性时间复杂度的方法求数论函数和。
例如, Min_25可用于求 10^{11} 内质数和或质数平方和。

P1835 素数密度

给定 L, R ，请计算区间 $[L, R]$ 中素数的个数。

数据范围： $1 \leq L \leq R < 2^{31}$ ， $R - L \leq 10^6$

关键在于 $R - L \leq 10^6$ ，直接用 $O(\sqrt{n})$ 判定质数也还是TLE。

预处理出 10^5 再利用筛出来的质数判断 $[L, R]$ 中的数是不是质数。 还是TLE

利用预处理出来的质数，采用埃氏筛的思想，将 $[L, R]$ 中的合数筛掉。

P4942 小凯的数字

小凯有一天突发奇想，写下了一串数字： $\overline{l(l+1)(l+2)\dots(r-1)r}$ 。

例如： $l=2, r=5$ 时，数字为：2345。

当 $l=8, r=12$ 时数字为：89101112。

小凯很喜欢数字 9，所以他想问你他写下的数字除以 9 的余数是多少？

注意到， $10^k \equiv 1 \pmod 9$

问题转化成 $(l + (l+1) + (l+2) + \dots + (r-1) + r) \% 9$ ，注意别爆 long long，每个询问 $O(1)$ 。

费马小定理

- 设 p 是素数，对于任意正整数 a 且 $p \nmid a$ ，都成立 $a^{p-1} \equiv 1 \pmod{p}$
- 设 p 是素数，对于任意正整数 a ，都成立 $a^p \equiv a \pmod{p}$

数学归纳法证明：

归纳起点 $1^p \equiv 1 \pmod{p}$ 。假设对于 $x \in \mathbb{N}$ 成立 $x^p \equiv x \pmod{p}$ ，需要证明 $x+1$ 也成立

$$(1+x)^p \equiv \binom{p}{0} + \binom{p}{1}x + \dots + \binom{p}{p}x^p \equiv 1 + x^p \equiv 1 + x \pmod{p}$$

成立

逆元

对于非零整数 a, m ，如果存在 b 使得 $ab \equiv 1 \pmod{m}$ ，就称 b 是 a 在模 m 意义下的逆元。

- 什么时候会用到逆元？
- 在要求取模并且有除法运算时就需要用到逆元。
- 例如：在没有取模的情况下，我们计算 $6 \times 2 \div 3 = 4$ 。
- 但是在 $\text{mod } 7$ 意义下我们先运算 $6 \times 2 = 12, 12 \equiv 5 \pmod{7}$ ，下一步就需要计算 $5 \div 3$ 。
- 显然时除不尽的，但是按照原本的意义应该是能整除的，此时引入乘法逆元。
- 我们发现 $3 \times 5 = 15, 15 \equiv 1 \pmod{7}$ ，所以3在模7意义下的逆元就是5。
- 原式 $5 \div 3$ 等价于 $5 \times 5 = 25, 25 \equiv 4 \pmod{7}$ 与没有取模计算的结果一致。
- 如何计算逆元？当 p 是素数时， $x(x < p)$ 在模 p 意义下的逆元为 x^{p-2}

P1082 [NOIP 2012 提高组] 同余方程

求关于 x 的同余方程 $ax \equiv 1(\text{mod } b)$ 的最小正整数解。

对于 40% 的数据， $2 \leq b \leq 10^3$ ；

对于 60% 的数据， $2 \leq b \leq 5 \times 10^7$ ；

对于 100% 的数据， $2 \leq a, b \leq 2 \times 10^9$ 。

输入输出样例

输入 #1

复制

输出 #1

复制

3 10

7

刚学的费马小定理完全用不了

裴蜀定理

- 对于方程, 设 $a, b, c \in \mathbb{Z}$, a, b 不全为0, $ax + by = c$. 有整数解 (x, y) 当且仅当 $\gcd(a, b) \mid c$

记 $d = \gcd(a, b)$. 因为 $d \mid a, b$, 所以, 存在整数 u, v 使得 $a = du, b = dv$ 成立. 因此, 总有

$$ax + by = d(ux + vy).$$

这就说明 $d \mid ax + by$.

反过来, 需要说明存在 x, y 使得等式成立. 如果 a, b 之一是0, 不妨设 $b = 0$, 那么它们的最大公约数为 $d = a$, 显然有 $(x, y) = (1, 0)$ 使得等式成立. 接下来, 考虑 a, b 均不为零的情形. 由于 $\gcd(a, b) = \gcd(-a, b) = \gcd(a, -b)$, 所以不妨设 a, b 都是正数.

考虑辗转相除法的过程, 有

$$\begin{aligned} a &= q_1 b + r_1, & 0 \leq r_1 < b, \\ b &= q_2 r_1 + r_2, & 0 \leq r_2 < r_1, \\ r_1 &= q_3 r_2 + r_3, & 0 \leq r_3 < r_2, \\ &\dots & \\ r_{n-3} &= q_{n-1} r_{n-2} + r_{n-1}, & 0 \leq r_{n-1} < r_{n-2}, \\ r_{n-2} &= q_n r_{n-1} + r_n, & 0 \leq r_n < r_{n-1}, \\ r_{n-1} &= q_{n+1} r_n. \end{aligned}$$

裴蜀定理

由于最大公约数是 d ，最后一步辗转相除时，一定有 $r_n = d$ 。所以，倒数第二个等式可以写作

$$d = r_n = r_{n-2} - q_n r_{n-1}.$$

从倒数第三个等式中解出

$$r_{n-1} = r_{n-3} - q_{n-1} r_{n-2}$$

再代入上式，就可以消去 r_{n-1} ：

$$\begin{aligned} d &= r_{n-2} - q_n(r_{n-3} - q_{n-1}r_{n-2}) \\ &= (1 + q_n q_{n-1})r_{n-2} - q_n r_{n-3}. \end{aligned}$$

类似地，可以逐步地消去所有 $r_{n-2}, r_{n-3}, \dots, r_2, r_1$ ，最终得到

$$d = xa + yb.$$

这就证明了存在 x, y 使得 $ax + by = d$ 成立。由前文分析可知，这也证明了原命题。

扩展欧几里得算法

- 对于方程，设 $a, b, c \in \mathbb{Z}$ ， a, b 不全为0， $ax + by = c$ 。有整数解 (x, y) 当且仅当 $\gcd(a, b) \mid c$
- 当 a, b 互素时， $ax + by = 1$ ，因此 $ax \equiv 1 \pmod{b}$ ，此时得到的 x 即 a 在模 b 意义下的逆元。

$$ax_1 + by_1 = \gcd(a, b)$$

$$bx_2 + (a \bmod b)y_2 = \gcd(b, a \bmod b)$$

由欧几里得定理可知： $\gcd(a, b) = \gcd(b, a \bmod b)$

$$\text{所以 } ax_1 + by_1 = bx_2 + (a \bmod b)y_2$$

$$\text{又因为 } a \bmod b = a - (\lfloor \frac{a}{b} \rfloor \times b)$$

$$\text{所以 } ax_1 + by_1 = bx_2 + (a - (\lfloor \frac{a}{b} \rfloor \times b))y_2$$

$$ax_1 + by_1 = ay_2 + bx_2 - \lfloor \frac{a}{b} \rfloor \times by_2 = ay_2 + b(x_2 - \lfloor \frac{a}{b} \rfloor y_2)$$

$$\text{因为 } a = a, b = b, \text{ 所以 } x_1 = y_2, y_1 = x_2 - \lfloor \frac{a}{b} \rfloor y_2$$

将 x_2, y_2 不断代入递归求解直至 b 为0递归 $x = 1, y = 0$ 回去求解。

```
int exgcd(int a, int b, int &x, int &y){
    if (!b) {
        x = 1, y = 0;
        return a;
    }
    int d = exgcd(b, a%b, x, y), t = x;
    x = y;
    y = t - (a / b) * y;
    return d;
}
```

P2054 [AHOI2005] 洗牌

对于扑克牌的一次洗牌是这样定义的，将一叠 N （ N 为偶数）张扑克牌平均分成上下两叠，取下面一叠的第一张作为新的一叠的第一张，然后取上面一叠的第一张作为新的一叠的第二张，再取下面一叠的第二张作为新的一叠的第三张.....如此交替直到所有的牌取完。

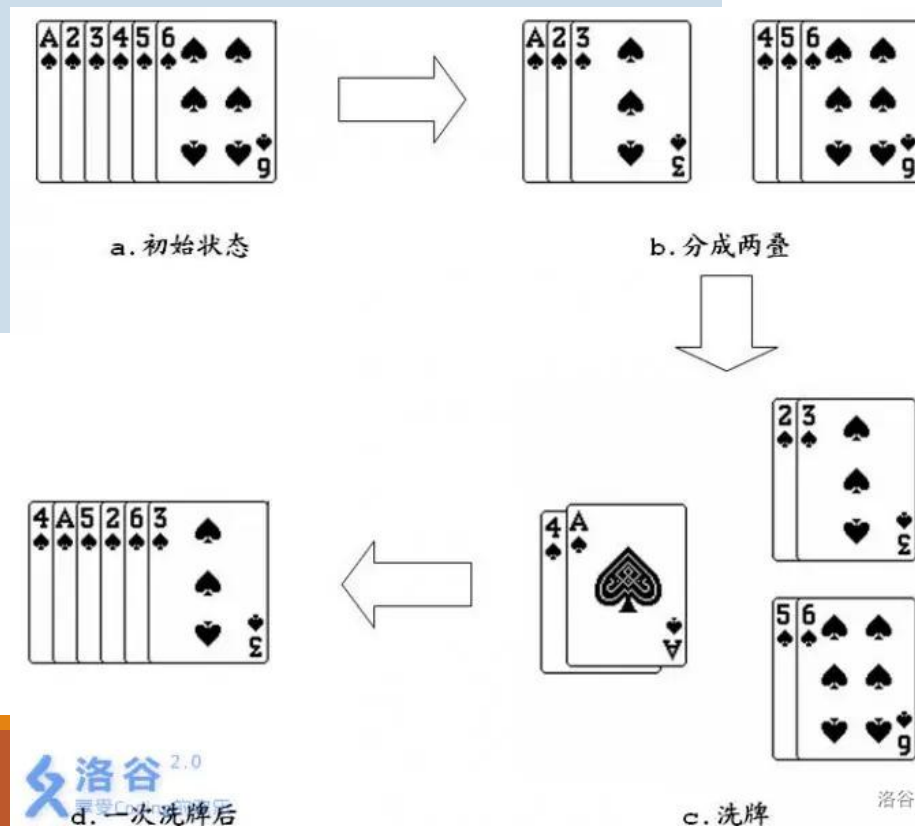
如果对一叠 6 张的扑克牌 1,2,3,4,5,6，进行一次洗牌的过程如下图所示：

经过一次洗牌，序列 1,2,3,4,5,6 变为 4,1,5,2,6,3。

当然，再对得到的序列进行一次洗牌，又会变为 2,4,6,1,3,5。

请问给定长度为 N 的扑克牌，洗牌 M 次后，第 L 张是多少？

$1 \leq N \leq 10^{10}$ ， $0 \leq M \leq 10^{10}$ ，且 N 为偶数。



P2054 [AHOI2005] 洗牌

对于初始序列[1,2,3,4,5,6]，考虑每个数去了哪里。

发现 [2,4,6,1,3,5]，位置为 i 的元素，洗牌一次移动到的位置与 $2i$ 有关。

具体的，每个数会移动到 $2i \bmod (n + 1)$ 的位置。

考虑移动 m 次，那么每个数会移动到 $2^m i$ ，现在要找到一个 i ，满足 $2^m i \equiv l \bmod (n + 1)$ 。

相当于 $2^m \frac{i}{l} \equiv 1 \bmod (n + 1)$ ，求解不定方程： $2^m x + (n + 1)y = 1$ 的解。

求出来的 x 乘 l 即为答案 i 。主要模数很大，乘法时需要慢慢乘（做加法）或者int128。